

Sistema de Detección de Ocupación de Parqueaderos

Grupo 4 - Los Perspectivistas

Juan David Buitrago Salazar - jbuitragosa@unal.edu.co

Juan David Cardenas Galvis - jcardenasgal@unal.edu.co

Juan Felipe Fajardo Garzón - jfajardoga@unal.edu.co

Camilo Andrés Medina Sánchez - cmedinasa@unal.edu.co

Nicolas Rodriguez Pirabán - nirodriguezpi@unal.edu.co

Docentes

Aura Maria Forero Pachón

Universidad Nacional de Colombia

Facultad de Ingeniería

Departamento de Ingeniería de Sistemas e Industrial

Bogotá D.C., Colombia

2026

Contenido

Nota sobre esta versión del documento	4
1. Resumen Ejecutivo	5
2. Descripción del Problema	6
3. Objetivos	7
3.1. Objetivo General	7
3.2. Objetivos Especificos y estado de cumplimiento	7
4. Alcance del Proyecto	9
4.1. Elementos Incluidos en el Alcance (implementados)	9
4.2. Elementos Excluidos / Trabajo Futuro (Fase 2)	10
5. Propuesta de Valor	11
6. Descripción Técnica	13
6.1. Arquitectura General	13
6.2. Modelo de Detección	14
6.3. Stack Tecnológico	14
6.4. API y Especificaciones de Entrada y Salida	15
6.5. Geometría del Lote y Navegación	16
7. Métricas de Desempeño	18
8. Cronograma de Desarrollo	19
9. Recursos Requeridos	20

9.1. Equipo de Trabajo.....	20
9.2. Infraestructura Tecnológica	20
10. Modelo de Monetización	21
10.1. Opciones de Comercialización	21
10.2. Segmentos de Clientes Objetivo	21
11. Ventajas Competitivas	22
12. Análisis de Riesgos	23
13. Plan de Acción Inmediato	24
14. Referencias.....	25

Nota sobre esta versión del documento

Este documento parte de la propuesta original del proyecto, redactada antes de iniciar el desarrollo, y la actualiza para reflejar el sistema efectivamente construido. Se conservan la estructura, el problema y la visión comercial, pero las secciones técnicas (arquitectura, modelo de detección, stack, especificaciones, métricas y cronograma) fueron revisadas para describir lo realmente implementado. Donde una decisión de diseño cambio respecto a la propuesta inicial, se indica explícitamente el cambio y su justificación. El sistema construido es un prototipo funcional (MVP) que integra detección por visión artificial, asignación de vehículos a espacios, calculo de ruta optima, reservas y una visualización 3D en tiempo real.

1. Resumen Ejecutivo

El presente documento expone el diseño y la implementación de un sistema de visión artificial orientado a la detección de la ocupación de espacios en parqueaderos, con aplicación en edificios de oficinas, centros comerciales, conjuntos residenciales y estacionamientos privados de la ciudad de Bogotá. El sistema construido emplea un modelo de detección de objetos basado en Faster R-CNN con backbone ResNet50-FPN para identificar los vehículos a partir de una imagen del parqueadero, los proyecta a coordenadas reales del lote mediante una transformación de perspectiva (homografía) y los asigna al espacio más cercano de una malla de 62 cupos. Los estados resultantes (ocupado, libre o reservado) se exponen a través de una API REST construida con FastAPI y se transmiten en tiempo real al cliente mediante eventos Server-Sent Events (SSE). El frontend, desarrollado en React con Three.js, presenta un tablero de control y una representación tridimensional interactiva del parqueadero que dibuja la ocupación y la ruta óptima hacia el espacio libre más cercano desde cualquiera de las dos entradas laterales. El prototipo demuestra de extremo a extremo el flujo detección -> asignación -> visualización -> reserva, y constituye la base de un producto escalable para operadores de estacionamiento.

El potencial de impacto se mantiene respecto a la propuesta inicial: las soluciones de estacionamiento inteligente reportan reducciones del 30 % al 50 % en el tiempo de búsqueda de cupo e incrementos del 24 % al 31 % en los ingresos de los operadores.

Palabras clave: visión artificial, detección de ocupación, parqueaderos inteligentes, Faster R-CNN, homografía, FastAPI, Three.js, computación visual.

2. Descripción del Problema

La gestión de parqueaderos en Bogotá y en las principales ciudades colombianas depende en la actualidad de métodos operativos que resultan insuficientes frente a la demanda y las expectativas de los usuarios contemporáneos. Los estacionamientos operan predominantemente mediante personal vigilante que estima la disponibilidad de manera subjetiva, complementado por señalización estática que no refleja el estado real de los espacios en tiempo real. Esta situación genera ineficiencias: los usuarios invierten tiempo considerable en la búsqueda de cupos, lo que incrementa el estrés y las emisiones vehiculares dentro de los recintos; los operadores subutilizan la capacidad instalada por falta de visibilidad sobre los patrones de ocupación; y la toma de decisiones gerenciales carece de datos históricos confiables sobre horarios pico y tendencias de uso (Viso AI, 2025; Parkade, 2024).

La magnitud de esta problemática adquiere relevancia si se considera que las soluciones de estacionamiento inteligente ya han demostrado un retorno de inversión comprobado en mercados internacionales, con incrementos de entre el 24 % y el 31 % en los ingresos por parqueadero y reducciones de entre el 30 % y el 50 % en el tiempo promedio de búsqueda de cupo (Parkade, 2024; ParkHelp, 2025). La tecnología de visión artificial que sustenta estos avances es accesible, escalable y puede integrarse sobre la infraestructura de cámaras de vigilancia existente, lo cual reduce la barrera de inversión inicial en hardware especializado (Viso AI, 2025; Ultralytics, 2026).

3. Objetivos

3.1. Objetivo General

Desarrollar un prototipo mínimo viable (MVP) de sistema de detección de ocupación de parqueaderos mediante técnicas de computación visual, funcional y demostrable de extremo a extremo, que sirva como base para una oferta comercial piloto dirigida a operadores de estacionamientos en Bogotá.

3.2. Objetivos Especificos y estado de cumplimiento

A continuación se listan los objetivos específicos definidos originalmente, indicando su estado tras la implementación del MVP.

A. Desarrollar un algoritmo de visión artificial para la detección de vehículos.

Implementado mediante un modelo Faster R-CNN ResNet50-FPN entrenado para distinguir espacios vacíos y ocupados. (Cumplido)

B. Implementar un sistema capaz de estimar la disponibilidad y ubicación de los espacios de estacionamiento. Implementado: el sistema asigna cada vehículo detectado al espacio más cercano de la malla y reporta cupos libres, ocupados y reservados. (Cumplido)

C. Implementar un pipeline de ingesta de imágenes y el módulo de procesamiento para la detección de vehículos mediante modelos preentrenados. Implementado sobre imágenes del parqueadero; la ingesta de video RTSP en vivo queda como trabajo futuro. (Cumplido parcialmente)

D. Desarrollar un mecanismo de mapeo de la distribución espacial de los espacios mediante coordenadas. Implementado: la malla de 62 espacios se define en

- coordenadas del mundo real compartidas entre backend y frontend, y los vehículos se proyectan a esas coordenadas mediante homografía. (Cumplido)
- E.** Diseñar un módulo de clasificación que determine el estado de cada espacio (ocupado, libre o reservado). Implementado mediante asignación por vecino más cercano y un gestor de reservas. (Cumplido)
- F.** Construir un tablero de control en tiempo real con mapa de ocupación, contadores y última actualización. Implementado en React con actualización por SSE y una vista 3D interactiva en Three.js. (Cumplido)
- G.** Implementar un módulo de historial y reportes con exportación en CSV y PDF. Parcial: el historial de ocupación se grafica en el cliente; la exportación a CSV/PDF queda como trabajo futuro. (Pendiente)

4. Alcance del Proyecto

4.1. Elementos Incluidos en el Alcance (implementados)

El prototipo construido contempla los siguientes elementos:

- Modelo de detección de objetos (Faster R-CNN ResNet50-FPN) entrenado sobre el dataset público PKLot para distinguir espacios vacíos y ocupados.
- Pipeline de procesamiento de imagen con definición de una región de interés (ROI), transformación de perspectiva (homografía) con OpenCV y normalización a 640x640 px para alimentar el detector.
- Proyección de las detecciones a coordenadas reales del lote (76,0 m x 13,5 m) y asignación de cada vehículo al espacio más cercano de una malla de 62 cupos.
- Servicio backend en FastAPI que expone el estado de los espacios, calcula la ruta optima y gestiona reservas temporales.
- Transmisión del estado en tiempo real mediante Server-Sent Events (SSE), con cache de detección para limitar el costo de inferencia.
- Interfaz web en React con tablero de control, gráfico de ocupación y una escena 3D interactiva del parqueadero construida con Three.js.
- Calculo y visualización de la ruta optima hacia el espacio libre más cercano desde dos entradas laterales (oeste y este).
- Flujo de reserva de un espacio desde las vistas de entrada, con liberación automática por expiración.

4.2. Elementos Excluidos / Trabajo Futuro (Fase 2)

Quedan fuera del alcance del MVP actual, y se reservan para iteraciones posteriores, los siguientes elementos:

- Ingesta de video en vivo por RTSP desde cámaras CCTV/IP (el MVP opera sobre imágenes del parqueadero).
- Persistencia en base de datos (SQLite/PostgreSQL); el estado y las reservas se mantienen actualmente en memoria.
- Exportación de reportes en CSV y PDF, y alertas automáticas por correo electrónico ante ocupación crítica.
- Integración con sistemas de pago o barreras automáticas de acceso.
- Detección de placas e identificación de usuarios.
- Soporte simultaneo de multiples cámaras y multiples ubicaciones, y analítica predictiva avanzada.

5. Propuesta de Valor

La propuesta de valor del sistema se articula en torno a cinco dimensiones de beneficio cuantificadas en la literatura especializada y en implementaciones previas de soluciones análogas. En primer lugar, se proyecta una reducción del 30 % al 50 % en el tiempo que el usuario invierte en la búsqueda de cupo disponible (Viso AI, 2025; Parkade, 2024). En segundo lugar, la visibilidad en tiempo real y la posibilidad de implementar tarifas dinamicas se asocian con incrementos del 24 % al 31 % en los ingresos del operador (Parkade, 2024; ParkHelp, 2025). En tercer lugar, la automatización del monitoreo reduce la dependencia de personal vigilante (ParkHelp, 2025; Ultralytics, 2026). En cuarto lugar, los usuarios finales acceden de manera instantanea a la información de disponibilidad y a la ruta hacia el cupo libre desde sus dispositivos (ParkHelp, 2025). Por ultimo, los operadores obtienen métricas operativas que les permiten tomar decisiones fundamentadas en datos (Parkade, 2024).

Tabla 1

Resumen de Beneficios Cuantificables del Sistema Propuesto

Beneficio	Cuantificación estimada	Fuente
Reducción en tiempo de búsqueda de cupo	30-50 %	Viso AI (2025); Parkade (2024)
Incremento en ingresos del operador	24-31 %	Parkade (2024); ParkHelp (2025)
Reducción de personal operativo manual	Menor dependencia de vigilantes	ParkHelp (2025); Ultralytics (2026)
Mejora en experiencia del usuario	Acceso instantaneo y ruta al cupo	ParkHelp (2025)
Disponibilidad de datos operativos	Métricas horarias, picos, patrones	Parkade (2024)

Nota. Los rangos de cuantificación corresponden a valores reportados en implementaciones de sistemas de estacionamiento inteligente en mercados internacionales.

6. Descripción Técnica

6.1. Arquitectura General

La arquitectura del sistema sigue un modelo de procesamiento en capas con separación de responsabilidades. En la capa de captura, el sistema toma una imagen del parqueadero (en el MVP, una imagen de prueba; en producción, un fotograma proveniente de una cámara IP/CCTV). El módulo de detección define una región de interés (ROI) sobre la zona de parqueo y aplica una transformación de perspectiva (homografía) para rectificar la vista, normalizando la imagen a 640x640 px. Sobre esa imagen, un detector Faster R-CNN ResNet50-FPN identifica los vehículos y los espacios ocupados.

Las coordenadas de las detecciones, expresadas en píxeles, se convierten a coordenadas reales del lote (76,0 m de ancho por 13,5 m de fondo) usando los factores de escala derivados de la homografía. Un módulo de asignación vincula cada vehículo detectado con el espacio más cercano de una malla de 62 cupos definida en coordenadas del mundo, empleando distancia euclidiana. El estado resultante (ocupado, libre, reservado) se expone mediante una API REST en FastAPI y se transmite a los clientes a través de un canal Server-Sent Events (SSE), que solo emite cuando cambia el estado de ocupación o de reservas. Para acotar el costo de inferencia, el detector mantiene un cache de 5 segundos.

Finalmente, el tablero de control web consume este flujo para presentar al usuario un mapa interactivo en 3D, contadores de cupos, un gráfico de ocupación histórica y la ruta optima hacia el espacio libre más cercano (Ultralytics, 2026; IJISRT, 2025).

Cambio respecto a la propuesta: la transformación geométrica de coordenadas cámara -> mundo, que en la propuesta se planteaba como un paso posterior, fue integrada como parte central del pipeline mediante homografía con OpenCV.

6.2. Modelo de Detección

El componente de detección se implementó con Faster R-CNN y backbone ResNet50-FPN de torchvision, con la cabeza de predicción ajustada a tres clases (fondo, espacio vacío y espacio ocupado). El modelo se entrenó sobre el dataset público PKLot, anotado en formato COCO, que contiene imágenes de parqueaderos bajo distintas condiciones de iluminación y clima. En inferencia se conservan únicamente las detecciones con confianza superior a 0,5 y se consideran ocupados los espacios con la etiqueta correspondiente. El peso entrenado se distribuye como un único archivo (parking_lot_detector.pth).

Cambio respecto a la propuesta: la propuesta inicial planteaba YOLOv8 como modelo principal. Durante el desarrollo se exploró YOLOv8 con el dataset Kaggle de detección de carros, pero el modelo finalmente desplegado es Faster R-CNN ResNet50-FPN entrenado sobre PKLot, por su buen desempeño en la distinción vacío/ocupado bajo la vista cenital del parqueadero.

6.3. Stack Tecnológico

El backend se desarrolló en Python (3.12+), con FastAPI como framework de servicios web y Uvicorn como servidor ASGI. El módulo de visión artificial utiliza PyTorch y torchvision (Faster R-CNN ResNet50-FPN) junto con OpenCV y NumPy para la transformación de perspectiva y el procesamiento de imagen. La comunicación en tiempo real se resuelve con Server-Sent Events sobre HTTP. El frontend se construyó en

React 19 con Vite como herramienta de build, y la visualización 3D se realiza con Three.js a través de React Three Fiber y drei. El enrutamiento entre vistas (panel de administración y vistas de entrada oeste/este) se maneja con un esquema basado en el hash de la URL.

Cambios respecto a la propuesta: (i) se seleccionó FastAPI sobre Flask; (ii) Three.js pasó de ser una opción a evaluar a ser el motor de visualización principal; (iii) el frontend se implementó en React (no Vue); y (iv) la persistencia en SQLite/PostgreSQL aun no se incorpora: en el MVP el estado de ocupación y las reservas se mantienen en memoria.

Tabla 2

Stack Tecnológico Implementado

Capa	Tecnología
Lenguaje backend	Python 3.12+
API / servidor	FastAPI 0.116 + Uvicorn
Visión artificial	PyTorch + torchvision (Faster R-CNN ResNet50-FPN)
Procesamiento de imagen	OpenCV, NumPy
Tiempo real	Server-Sent Events (SSE)
Frontend	React 19 + Vite
Visualización 3D	Three.js (React Three Fiber + drei)
Persistencia (MVP)	En memoria (BD relacional planificada para Fase 2)

6.4. API y Especificaciones de Entrada y Salida

El sistema acepta como entrada imágenes del parqueadero; en el MVP se opera sobre imágenes de prueba, con una arquitectura preparada para recibir fotogramas de cámaras IP fijas (MP4/RTSP) en fases posteriores. La API REST expone los siguientes endpoints principales:

Tabla 3

Endpoints de la API

Metodo y ruta	Descripción
GET /health	Verificación de estado del servicio.
GET /espacios	Lista de los 62 espacios con su estado (ocupado/reservado).
GET /estado	Estado completo: espacios, destino sugerido y ruta optima.
GET /suscripción/estado	Canal SSE que emite el estado al cambiar ocupación o reservas.
GET /ruta-optima	Ruta optima hacia el espacio libre más cercano desde una entrada.
POST /reservar	Reserva el espacio libre más cercano a la entrada indicada.
DELETE /reservar/{id}	Libera una reserva (al estacionar o por cancelación).

Las salidas del sistema comprenden el tablero de control web con actualización en tiempo real, el mapa 3D de ocupación con la ruta optima resaltada, contadores de cupos libres/ocupados y un gráfico de ocupación. La exportación de reportes en CSV/PDF y las alertas por correo se contemplan para la Fase 2 (ParkHelp, 2025).

6.5. Geometría del Lote y Navegación

El parqueadero se modela como un lote de 76,0 m x 13,5 m con 62 espacios organizados en dos filas de 31 cupos cada una, separadas por un pasillo central de doble sentido. Cada fila se divide en dos bloques (14 + 17 cupos) con un espacio reservado para una isla/luminaria central. Esta geometría es compartida exactamente entre el backend y el frontend, de modo que las coordenadas del modelo de detección y las posiciones de la escena 3D siempre coinciden.

El sistema define dos entradas laterales, oeste ($x = -38$) y este ($x = +38$). El navegador calcula la ruta optima como una trayectoria de tres tramos: desde la entrada lateral hasta el pasillo central, a lo largo del pasillo hasta la columna del espacio objetivo,

y finalmente hasta el cupo. Entre todos los espacios libres se elige el que minimiza la longitud total de la ruta. El módulo de reservas, seguro ante concurrencia, marca temporalmente un espacio como reservado (con expiración a los 60 segundos) y versiona cada cambio para invalidar el estado en los clientes conectados.

7. Métricas de Desempeño

El desempeño del sistema se evalúa conforme a los criterios de la Tabla 4, que establece metas para el prototipo mínimo viable y para una versión de aceptación comercial. La columna de estado refleja la situación del MVP actual.

Tabla 4

Métricas de Desempeño del Sistema

Métrica	Meta MVP	Aceptación comercial	Estado actual
Precisión de detección	85-90 %	> 80 %	Modelo Faster R-CNN operativo; en validación
Latencia (captura a dashboard)	< 5 s	< 10 s	Cache de 5 s + SSE; dentro de meta
Disponibilidad del sistema	95 %	99 % (Fase 2)	MVP local
Cobertura espacial	100 % con max. 2 cámaras	Escalable a N cámaras	1 vista / 62 espacios
Tiempo de configuración inicial	< 2 h en sitio	< 4 h	Malla y ROI configurables
Falsos positivos	< 10 % / día	< 5 %	En medición

Nota. Las metas de aceptación comercial corresponden a estándares proyectados para la versión de producción, fuera del alcance del presente MVP.

8. Cronograma de Desarrollo

El proyecto se estructuro en cinco semanas de desarrollo iterativo. La Tabla 5 resume los hitos y su resultado tras la ejecución.

Tabla 5

Cronograma de Desarrollo y Resultados

Semana	Hito principal	Resultado
1	Construcción / selección del dataset	Uso del dataset público PKLot (formato COCO) para entrenar la detección vacío/ocupado.
2	Backend y modelo de detección	API en FastAPI y modelo Faster R-CNN ResNet50-FPN entrenado e integrado al pipeline.
3	Dashboard MVP	Interfaz React con mapa de ocupación, contadores y actualización en tiempo real vía SSE.
4	Integración y pruebas	Pipeline detección -> asignación -> ruta -> reserva integrado; pruebas sobre imágenes de muestra.
5	Refinamiento y demostración	Visualización 3D en Three.js, navegación por entradas y ajustes; demostración del flujo completo.

9. Recursos Requeridos

9.1. Equipo de Trabajo

El proyecto fue desarrollado por el Equipo del Grupo 4, con roles de ingeniería full stack (visión artificial, backend y frontend) y de validación de producto/negocio para la gestión de clientes y la presentación de demostraciones.

9.2. Infraestructura Tecnológica

Los recursos comprenden un servidor de desarrollo local (o una instancia equivalente a AWS EC2 t3.micro para despliegue) y almacenamiento mínimo para imágenes y pesos del modelo. El entrenamiento del detector se realizó con aceleración por GPU (CUDA cuando esta disponible). Todas las librerías y frameworks utilizados son de código abierto: PyTorch y torchvision, FastAPI, Uvicorn, OpenCV, NumPy, React, Vite y Three.js. La incorporación de una base de datos relacional (PostgreSQL o SQLite) se planifica para la Fase 2.

10. Modelo de Monetización

10.1. Opciones de Comercialización

Se plantean dos modalidades de comercialización. La primera es un modelo de pago único por parte del parqueadero donde se haga la implementación, con un precio de entre 2 y 3 millones de pesos colombianos (aproximadamente 500-750 USD), que incluye instalación, capacitación inicial y reportes durante el período de prueba, reteniendo el cliente la titularidad de los datos. La segunda es una suscripción mensual de entre 500 y 1.000 USD por parqueadero, que contempla mantenimiento, actualizaciones del modelo y soporte técnico, con escalamiento a múltiples cámaras por un costo adicional de 200 USD por cámara.

10.2. Segmentos de Clientes Objetivo

Los segmentos prioritarios para la comercialización inicial en Bogotá son: edificios de oficinas en sectores como Chapinero y Zona T (50-300 cupos); centros comerciales de mediano tamaño, con parqueaderos de superficie o subterráneos; conjuntos residenciales y condominios, donde el sistema representa una amenidad de valor agregado; y estacionamientos privados independientes que buscan maximizar el aprovechamiento de sus predios con inversión mínima en tecnología.

11. Ventajas Competitivas

El sistema presenta cinco ventajas competitivas. En primer lugar, su bajo costo de entrada, derivado de la compatibilidad con la infraestructura CCTV existente sin hardware especializado (Viso AI, 2025; Ultralytics, 2026). En segundo lugar, su corto tiempo de llegada al mercado: un MVP funcional en cinco semanas, frente a los tres a seis meses de las soluciones empresariales convencionales (ParkHelp, 2025). En tercer lugar, su adaptación al mercado local colombiano. En cuarto lugar, la flexibilidad de su modelo de despliegue, que permite operación on-premise sin dependencia de la nube. En quinto lugar, una arquitectura desacoplada (detección, asignación, navegación, reservas y visualización) diseñada para escalar a múltiples cámaras y ubicaciones en iteraciones posteriores (Ultralytics, 2026; IJISRT, 2025).

12. Análisis de Riesgos

La Tabla 6 presenta los principales riesgos del proyecto, junto con su probabilidad, impacto y las estrategias de mitigación.

Tabla 6

Matriz de Riesgos del Proyecto

Riesgo	Prob.	Impacto	Estrategia de mitigación
Precisión de detección baja bajo iluminación variable	Media	Alto	Entrenar con PKLot (múltiples condiciones); ajustar umbral y exposición.
Incompatibilidad con las cámaras del cliente	Baja	Medio	Verificar conectividad IP temprano; operar sobre imagen/video pregrabado como contingencia.
Modificaciones al layout del parqueadero	Baja	Bajo	Malla y ROI configurables para recalibración sencilla.
Perdida de estado por operar en memoria	Media	Medio	Migrar a base de datos persistente en la Fase 2.
Demanda de soporte insostenible tras el lanzamiento	Media	Medio	Documentación detallada, FAQ y guía de resolución de problemas.

13. Plan de Acción Inmediato

Con el MVP construido, las acciones inmediatas se orientan a la validación comercial y al cierre de las brechas técnicas pendientes. En el plano comercial: realizar entrevistas con operadores de parqueaderos en Bogotá para validar la disposición de pago, confirmar un cliente piloto con acceso a una cámara CCTV operativa y presentar una demostración formal del sistema (Precise Park Link, 2025; Our Parking Space, 2025). En el plano técnico: incorporar la ingesta de video RTSP en vivo, añadir persistencia en base de datos, implementar la exportación de reportes (CSV/PDF) y las alertas por correo, y validar las métricas de precisión y latencia sobre datos del parqueadero piloto.

14. Referencias

Ammar Nassan Alhajali. (2021). PKLot dataset [Conjunto de datos]. Kaggle.

<https://www.kaggle.com/datasets/ammarnassanalhajali/pklot-dataset>

IJSRT. (2025). Parking occupancy detection using computer vision techniques.

International Journal of Innovative Science and Research Technology.

<https://www.ijisrt.com/parking-occupancy-detection-using-computer-vision-techniques>

Our Parking Space. (2025). How to implement smart parking solutions for urban living:

A step-by-step guide. <https://www.ourparkingspace.com/post/how-to-implement-smart-parking-solutions-for-urban-living-a-step-by-step-guide>

Parkade. (2024). Return on investment of parking management software.

<https://parkade.com/post/roi-parking-management-software>

ParkHelp. (2025). 5 ways smart parking boosts ROI for retail spaces.

<https://www.parkhelp.com/5-ways-smart-parking-boosts-roi-for-retail-spaces/>

Precise Park Link. (2025). 4 factors to consider before deploying a smart parking

guidance system. <https://preciseparklink.com/news/4-factors-to-consider-before-deploying-a-smart-parking-guidance-system/>

Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object

detection with region proposal networks. Advances in Neural Information Processing Systems, 28.

Three.js. (2026). Three.js - JavaScript 3D library. <https://threejs.org/>

Ultralytics. (2026). Parking management using YOLOv8.

<https://docs.ultralytics.com/guides/parking-management/>

Viso AI. (2025). Real-time parking lot occupancy with AI visión.

<https://viso.ai/applications/parking-lot-occupancy-detection/>